

# React useEffect & Cleanup Function – Ausführliche Erklärung

Diese PDF erklärt Schritt für Schritt:

- useState
- useEffect
- Dependency Array
- Cleanup Function (return)
- Reihenfolge der Ausführung
- Warum Arrays und Objekte Effekte erneut auslösen

## Ausgangscode:

```
const [count, setCount] = useState(0);
const [arrayState, setArrayState] = useState([1]);
const [objectState, setObjectState] = useState({ attribute: "a" });

useEffect(() => {
  console.log(count);
  console.log(arrayState);
  console.log(objectState);

  return () => {
    console.log(count, "RETURNED EFFECT");
    console.log(arrayState, "RETURNED EFFECT");
    console.log(objectState, "RETURNED EFFECT");
  };
}, [count, arrayState, objectState]);
```

## 1. useState erklärt

useState speichert Daten innerhalb einer React-Komponente.

Beispiel:

```
const [count, setCount] = useState(0);
```

count:

→ aktueller Wert

setCount:

→ Funktion zum Ändern des Wertes

0:

→ Startwert

Wenn setCount(5) ausgeführt wird,  
dann wird count = 5.

## 2. Array und Objekt States

```
const [arrayState, setArrayState] = useState([1]);
const [objectState, setObjectState] = useState({ attribute: "a" });
```

Hier werden komplexe Datentypen gespeichert:

- Arrays
- Objekte

Diese Typen verhalten sich in React anders als Zahlen oder Strings.

### 3. useEffect erklärt

useEffect reagiert auf Änderungen.

Der Effekt läuft:

- Beim ersten Render
- Wenn sich eine Dependency ändert

Hier:

```
[count, arrayState, objectState]
```

bedeutet:

Der Effekt läuft erneut,  
wenn sich einer dieser Werte verändert.

### 4. Dependency Array

Dependency Arrays bestimmen,  
wann ein Effekt erneut ausgeführt wird.

Beispiel:

```
[count]
```

→ Effekt läuft nur wenn count sich ändert.

Leeres Array:

```
[]
```

→ Effekt läuft nur einmal beim ersten Render.

Kein Array:

```
useEffect(() => {})
```

→ Effekt läuft nach JEDEM Render.

### 5. Cleanup Function – Sehr ausführlich erklärt

Die Cleanup Function ist der wichtigste Teil dieses Beispiels.

Sie befindet sich hier:

```
return () => {  
  ...  
}
```

Viele Anfänger denken:

„Die Funktion läuft nach dem Effekt.“

Das stimmt NICHT.

Die Cleanup Function läuft:

1. Bevor der Effekt erneut ausgeführt wird
2. Wenn die Komponente entfernt wird (Unmount)

-----  
WICHTIGE REIHENFOLGE  
-----

1. Effekt läuft
2. State ändert sich
3. Cleanup vom ALTEN Effekt läuft
4. Neuer Effekt läuft

-----  
BEISPIEL  
-----

INITIALER RENDER:

```
console.log(count);
```

Ausgabe:

0

-----  
JETZT:

```
setCount(1)
```

-----  
WAS PASSIERT INTERN?

1. Cleanup vom ALTEN Effekt läuft

Ausgabe:

0 RETURNED EFFECT

Warum 0?

Weil die Cleanup Function die ALTEN Werte kennt.

Sie arbeitet mit den Werten,  
die existierten,  
als der Effekt erstellt wurde.

-----  
DANACH:

Der neue Effekt läuft.

Jetzt ist:

```
count = 1
```

Ausgabe:

1

-----  
WICHTIGES VERSTÄNDNIS

Cleanup bedeutet:

„Räume den alten Effekt auf,  
bevor ein neuer Effekt startet.“

-----  
WARUM BRAUCHT MAN CLEANUP?

Sehr wichtig bei:

- Event Listenern
- Timern
- Intervallen
- WebSockets
- API Verbindungen
- Subscriptions

-----  
BEISPIEL MIT EVENT LISTENER

```
useEffect(() => {  
  window.addEventListener("resize", handleResize);  
  
  return () => {  
    window.removeEventListener("resize", handleResize);  
  };  
}, []);
```

Ohne Cleanup:

→ Mehrere Listener würden sich ansammeln.

Das verursacht:

- Memory Leaks
- Langsame Apps
- Doppelte Events

-----  
BEISPIEL MIT setInterval  
-----

```
useEffect(() => {  
  const interval = setInterval(() => {  
    console.log("running");  
  }, 1000);  
  
  return () => {  
    clearInterval(interval);  
  };  
}, []);
```

Ohne Cleanup:

→ Das Interval würde weiterlaufen,  
selbst wenn die Komponente entfernt wurde.

## 6. Referenzvergleich

6. Warum Arrays und Objekte Effekte erneut auslösen

React vergleicht:

- Primitive Werte → per Inhalt
- Objekte/Arrays → per Referenz

-----  
Primitive Werte  
-----

1 === 1

→ true

-----  
Arrays  
-----

[1] === [1]

→ false

Warum?

Weil beide Arrays unterschiedliche Speicheradressen besitzen.

-----  
Das bedeutet:  
-----

```
setArrayState([1]);
```

löst den Effekt erneut aus.

Obwohl der Inhalt identisch ist.

Denn:

```
alterArray !== neuerArray
```

Dasselbe gilt für Objekte.

## Zusammenfassung

## ZUSAMMENFASSUNG

- useEffect reagiert auf Änderungen
- Dependency Arrays steuern wann Effekte laufen
- Cleanup Functions räumen alte Effekte auf
- Cleanup läuft VOR dem nächsten Effekt
- Arrays und Objekte werden über Referenzen verglichen
- Deshalb lösen neue Arrays/Objekte oft Re-Renders aus